Computer Programming Summer Interest Group

Session 10 / Files

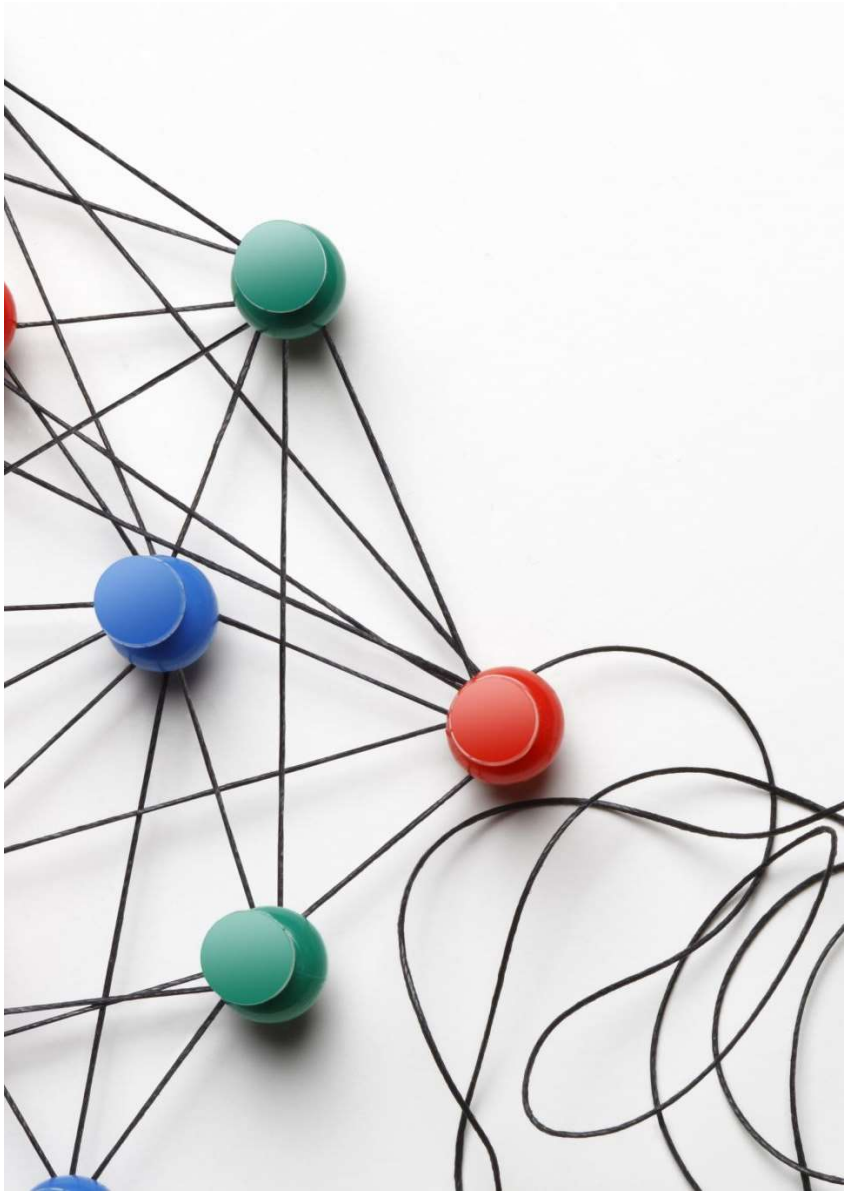
8/20/2025

John 20: 30-01

Now Jesus did many other signs in the presence of the disciples, which are not written in this book; but these are written so that you may believe that Jesus is the Christ, the Son of God, and that by believing you may have life in his name.

Objectives

- By the end of this session, you will be able to:
  - Understand the concept of file handling in Python.
  - Open, read, and write text files using Python.
  - Differentiate between modes:
    - **read ('r')**
    - **write ('w')**
    - **append ('a')**
    - **binary ('b')**
  - Use context managers (with statement) for safe file handling.



Different Types of Data Storage

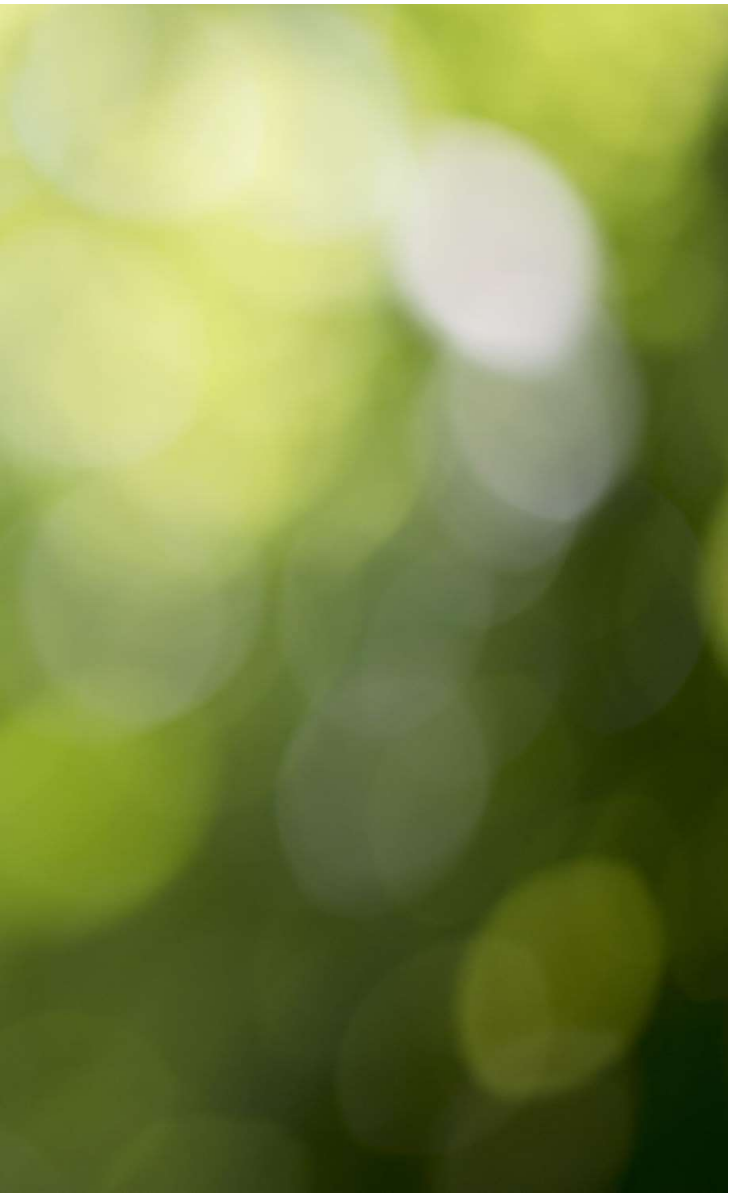
Registers	• Very Fast • Very Small • Not Persistent
Cache	• Fast • Small • Not Persistent
RAM	• Moderately Fast • Large • Not Persistent
File Storage	• Slow • Vast • Persistent

File Storage

- Allows data to be retained even while powered down
- Storage devices can be physically removed without data loss
- Can contain very large volumes of data
- Slow access times
- Much cheaper for unit of storage (i.e. $/Gb)

Implications for Programmers

- Programmers need to be able to
  - Read data from file systems
  - Write data to file systems
- Need to be able to read / write data in different file formats
  - Binary (e.g. JPEG, MP4)
  - Text based (e.g. YAML, JSON, CSV)
  - Application specific formats (e.g. MS Word, MS Excel)



What is a File System?

- The file system is the way the operating system (OS) organizes the data on the storage device.
- It allows the OS to save data to the storage device and retrieve it from the storage device.
- Windows and Mac use a hierarchical directory (aka folder) structure to organize files
- A directory can contain 0 or more files and / or 0 or more other directories
- All directories except the root directory have a parent directory
- The series of directories between the root directory and a file is called the file's path

Basic file output

- open – A function that “opens” a file for input or output operations
  - File name opened here is “file.txt”
  - It is open for “write” operations only
  - The encoding is “utf-8”, which is a standard for encoding data
  - It returns an object which actually performs I/O operations on the file
- out_file.write – Writes text to the file
- out_file.close – Closes the file for writing
  - Once closed the file object can no longer be used to perform I/O

```
out_file = open(
    file="file.txt",
    mode="w",
    encoding="utf-8")

out_file.write("Hello World!")
out_file.close()
```

Basic file input

- open – A function that “opens” a file for input or output operations
  - File name opened here is “file.txt”
  - It is open for “read” operations only
  - The encoding is “utf-8”, which is a standard for encoding data
  - It returns an object which actually performs I/O operations on the file
- in_file.read – Reads the entire file and returns its content as a string
- in_file.close – Closes the file for reading

```
in_file = open(
    file="file.txt",
    mode="r",
    encoding="utf-8")

text = in_file.read()
print(text)
in_file.close()
```

Basic file output – Multiple Lines

- Use the ‘\n’ to insert a new line in the file

```
out_file = open(
    file="lines.txt",
    mode="w",
    encoding="utf-8")

for line in range(10):
    out_file.write(f"Line {line}\n")

out_file.close()
```

Basic file input – Multiple Lines

- Use **`readline`** to read a single line in at a time
- The line read will be **None** when reading after end of file.
  - We use this to exit from the loop
- Each line will have a '\n' at the end which will create a line feed when printing out to the console
- The **`end=""`** in the print suppresses the automatic line feed used in print.

```
file="lines.txt",
    mode="r",
    encoding="utf-8")

lines = []
while True:
    line = in_file.readline()
    if not line:
        break
    lines.append(line)
in_file.close()

for line in lines:
    print(line, end="")
```

Basic file output – Appending to the end of a file

- Use **mode = 'a'** to append to the end of a file when opening.

```
out_file = open(file="timestamps.txt",
                mode="a",
                encoding="utf-8")
ts = time.localtime()
out_file.write(
    f"{ts.tm_year}-"
    f"{ts.tm_mon:02}-"
    f"{ts.tm_mday:02} "
    f"{ts.tm_hour:02}:"
    f"{ts.tm_min:02}:"
    f"{ts.tm_sec:02}\n")
out_file.close()
```

Basic file input – Reading all lines of a file

- The **`readlines`** method does essentially what the loop does in the previous example.

```
in_file = open(
    file="lines.txt",
    mode="r",
    encoding="utf-8")

lines = in_file.readlines()
in_file.close()

for line in lines:
    print(line, end="")
```

Basic file output – Binary data

- Use **mode = ‘wb’** to write binary content.
- Generally smaller size files
- Not generally human readable
- Need to convert content into a “bytearray”

```
import struct
import math

# Convert a floating-point value to a “byte array”
packed_bytes = struct.pack("d", math.pi)
byte_array = bytearray(packed_bytes)

# Open the file for writing and write the content
out_file = open(file="binary.bin", mode="wb")
out_file.write(byte_array)
out_file.close()
```

Basic file input – Binary data

- Use **mode = ‘rb’** to read binary content.
- Reads in raw byte values
- Need to unpack back into desired datatype for use

```
import struct

# Read the binary content from the file
in_file = open(file="binary.bin", mode="rb")
byte_array = in_file.read()
in_file.close()

# Convert it back into a float
unpacked_value = struct.unpack("d", byte_array)[0]
print(unpacked_value)
```

Context Managers

- It's generally better to use a "context manager" when performing file I/O
- The context manager will ensure the file is closed after leaving scope

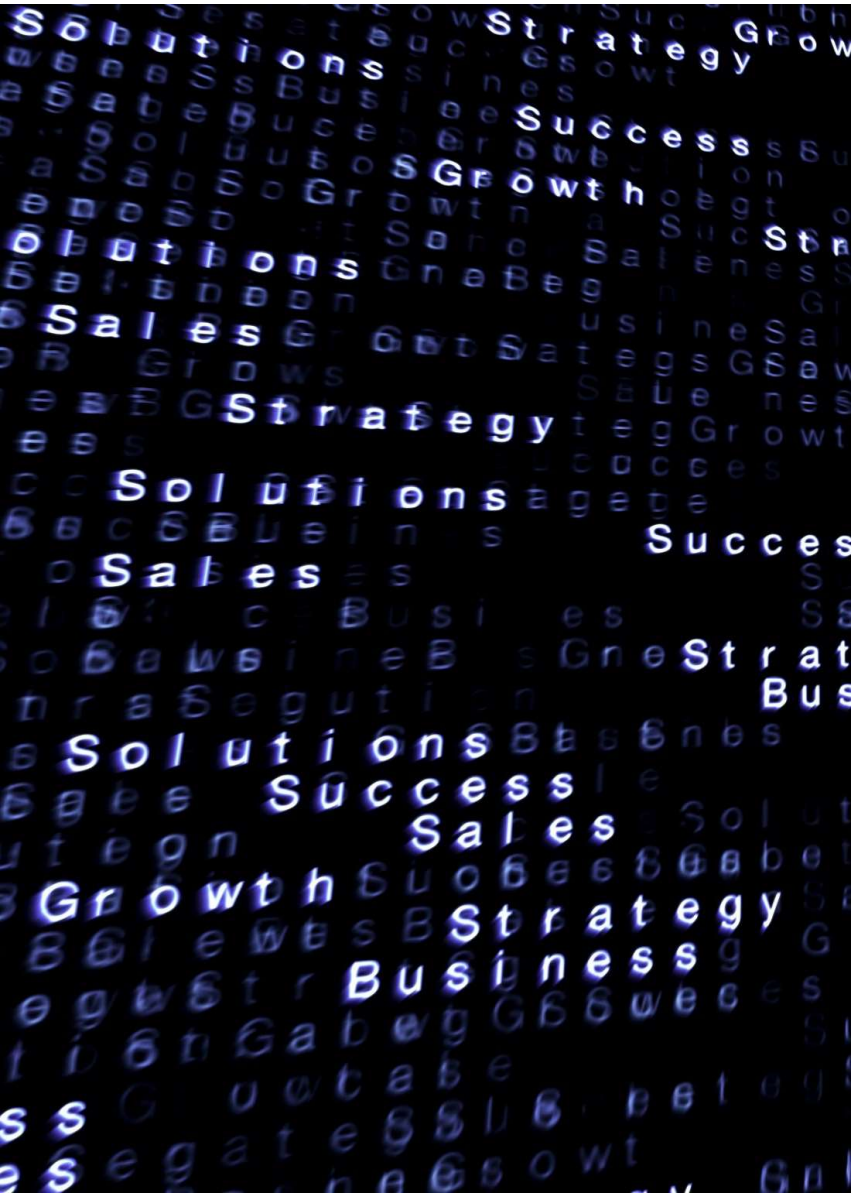
```
with open(file="timestamps.txt",
          mode="a",
          encoding="utf-8") as out_file:
    ts = time.localtime()
    out_file.write(
        f"{ts.tm_year}-"
        f"{ts.tm_mon:02}-"
        f"{ts.tm_mday:02} "
        f"{ts.tm_hour:02}:"
        f"{ts.tm_min:02}:"
        f"{ts.tm_sec:02}\n")
```

Context Managers

- Context managers can be used for both write and read operations

```
with open(file="timestamps.txt",
          mode="r",
          encoding="utf-8") as in_file:
    lines = in_file.readlines()

for line in lines:
    print(line, end="")
```



Activities

- Write a program that creates a copy of itself using the techniques demonstrated here.
  - Use the ‘os’ package
  - os.path.abspath(__file__) – the full path to the running program
- In your program display to the console:
  - The number of lines copied
  - The number of characters copied